

```

import java.io.*;
import java.util.*;

public class LRU {
    public static void main(String[] args) throws IOException {

        BufferedReader reader = new BufferedReader(new
InputStreamReader(System.in));

        // Input variables
        int totalFrames;           // Number of available frames in
memory
        int pointer = 0;           // Keeps track of which frame to
replace next
        int hitCount = 0;          // Number of page hits
        int faultCount = 0;        // Number of page faults
        int referenceLength;       // Length of the reference string
        boolean isMemoryFull = false;

        // Data structures
        int[] frameBuffer;         // Stores current pages in memory
        int[] referenceString;     // Stores all pages to be referenced
        int[][] memoryLayout;     // Stores memory state after each
reference
        ArrayList<Integer> recentPages = new ArrayList<>(); // Keeps
pages in order of use (for LRU)

        // Input from user
        System.out.print("Enter number of Frames: ");
        totalFrames = Integer.parseInt(reader.readLine());

        System.out.print("Enter length of Reference String: ");
        referenceLength = Integer.parseInt(reader.readLine());

        referenceString = new int[referenceLength];
        memoryLayout = new int[referenceLength][totalFrames];
        frameBuffer = new int[totalFrames];

        // Initialize all frames as empty (-1)
        for (int i = 0; i < totalFrames; i++)
            frameBuffer[i] = -1;

        System.out.println("Enter the Reference String: ");
        for (int i = 0; i < referenceLength; i++) {
            referenceString[i] = Integer.parseInt(reader.readLine());
        }

        System.out.println();

        // Start processing each page in the reference string
        for (int i = 0; i < referenceLength; i++) {

            // If page is already in stack (recently used list), remove
it
            if (recentPages.contains(referenceString[i])) {

                recentPages.remove(recentPages.indexOf(referenceString[i]));
            }
        }
    }
}

```

```

// Add current page to the top (most recently used)
recentPages.add(referenceString[i]);

int pageFoundAt = -1;

// Check if page is already in one of the frames (HIT)
for (int j = 0; j < totalFrames; j++) {
    if (frameBuffer[j] == referenceString[i]) {
        pageFoundAt = j;
        hitCount++;
        break;
    }
}

// If page not found ? Page Fault occurs
if (pageFoundAt == -1) {
    // If memory already full, find least recently used page
    to replace
    if (isMemoryFull) {
        int leastRecentlyUsedIndex = referenceLength; //
        large initial value
        for (int j = 0; j < totalFrames; j++) {
            if (recentPages.contains(frameBuffer[j])) {
                int indexInStack =
                recentPages.indexOf(frameBuffer[j]);
                if (indexInStack < leastRecentlyUsedIndex) {
                    leastRecentlyUsedIndex = indexInStack;
                    pointer = j; // frame to replace
                }
            }
        }

        // Replace the selected frame with new page
        frameBuffer[pointer] = referenceString[i];
        faultCount++;
        pointer++;

        // If pointer exceeds frame count, reset and mark memory
        as full
        if (pointer == totalFrames) {
            pointer = 0;
            isMemoryFull = true;
        }
    }

    // Store memory layout for this step
    for (int j = 0; j < totalFrames; j++)
        memoryLayout[i][j] = frameBuffer[j];
}

// Display memory layout (each row = frame, each column = step)
System.out.println("\nMemory Layout:");
for (int i = 0; i < totalFrames; i++) {
    for (int j = 0; j < referenceLength; j++) {
        System.out.printf("%3d ", memoryLayout[j][i]);
    }
}

```

```

        System.out.println();
    }

    // Print final statistics
    System.out.println("\nTotal Page Hits: " + hitCount);
    System.out.println("Total Page Faults: " + faultCount);
    System.out.println("Hit Ratio: " + (float) hitCount /
referenceLength);
    }
}

import java.io.*;
import java.util.*;
public class LRU {
public static void main(String[] args) throws IOException
{
BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
int frames,pointer = 0, hit = 0, fault = 0,ref_len;
Boolean isFull = false;
int buffer[];
ArrayList<Integer> stack = new ArrayList<Integer>();
int reference[];
int mem_layout[][];
System.out.println("Please enter the number of Frames: ");
frames = Integer.parseInt(br.readLine());
System.out.println("Please enter the length of the Reference string: ");
ref_len = Integer.parseInt(br.readLine());
reference = new int[ref_len];
mem_layout = new int[ref_len][frames];
buffer = new int[frames];
for(int j = 0; j < frames; j++)
buffer[j] = -1;
System.out.println("Please enter the reference string: ");
for(int i = 0; i < ref_len; i++)
{
reference[i] = Integer.parseInt(br.readLine());
}
System.out.println();
for(int i = 0; i < ref_len; i++)
{
if(stack.contains(reference[i]))
{
stack.remove(stack.indexOf(reference[i]));
}
stack.add(reference[i]);
int search = -1;
for(int j = 0; j < frames; j++)
{
if(buffer[j] == reference[i])
{
search = j;
hit++;
break;
}
}
if(search == -1)
{

```

```

if(isFull)
{
    int min_loc = ref_len;
    for(int j = 0; j < frames; j++)
    {
        if(stack.contains(buffer[j]))
        {
            int temp = stack.indexOf(buffer[j]);
            if(temp < min_loc)
            {
                min_loc = temp;
                pointer = j;
            }
        }
    }
    buffer[pointer] = reference[i];
    fault++;
    pointer++;
    if(pointer == frames)
    {
        pointer = 0;
        isFull = true;
    }
    for(int j = 0; j < frames; j++)
        mem_layout[i][j] = buffer[j];
    }
    for(int i = 0; i < frames; i++)
    {
        for(int j = 0; j < ref_len; j++)
            System.out.printf("%3d ", mem_layout[j][i]);
        System.out.println();
    }
    System.out.println("The number of Hits: " + hit);
    System.out.println("Hit Ratio: " + (float)((float)hit/ref_len));
    System.out.println("The number of Faults: " + fault);
}
}

```